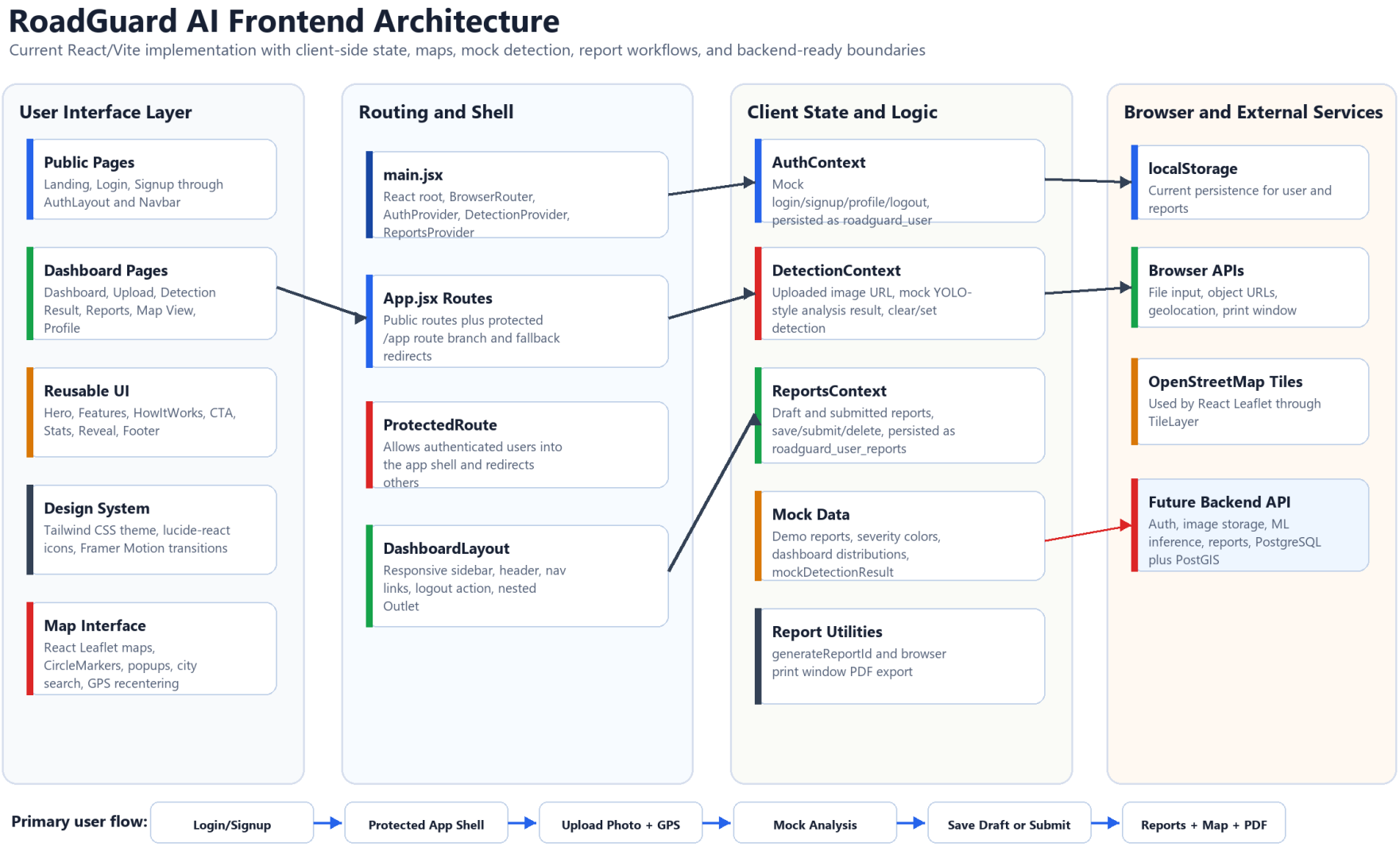
RoadGuard AI Frontend Architecture

Architecture diagram and implementation notes for the current React frontend

This document explains how the RoadGuard AI frontend is structured today. It covers routing, page composition, shared state, browser storage, maps, report actions, and the realistic backend boundary for adding PostgreSQL with PostGIS later.



Current Frontend Layers

Layer	Implementation	Responsibility
Application boot	src/main.jsx	Mounts React, enables BrowserRouter, and wraps App with Auth, Detection, and Reports providers.
Routing	src/App.jsx	Defines public routes, protected app routes, nested dashboard routes, and redirects.
Layout shell	DashboardLayout and AuthLayout	Provides dashboard navigation, responsive sidebar, header controls, auth page framing, and nested content outlet.
Feature pages	Dashboard, Upload, DetectionResult, Reports, MapView, Profile	Implements the citizen reporting workflow, map view, profile management, and report browsing.
Shared UI	Hero, Navbar, Footer, CTA, Features, HowItWorks, Stats, Reveal	Supports the landing experience and repeated presentation sections.
Styling and motion	Tailwind CSS, Framer Motion, lucide-react	Controls the visual system, responsive layout, animation, and iconography.

Primary Data Flow

Current flow: A user signs in through the mock AuthContext, enters the protected app shell, uploads or captures a road image, attaches GPS through the browser geolocation API, runs the mock analysis flow, then saves a draft or submits the report through ReportsContext.

Step	Frontend module	State or output
Authenticate	Login.jsx, Signup.jsx, AuthContext	Creates a mock user and persists roadguard_user in localStorage.
Capture evidence	Upload.jsx	Stores a browser object URL for the selected image and requests GPS coordinates.
Analyze	DetectionContext, mockData	Combines mockDetectionResult with image, report id, GPS, severity, score, and priority.
Review result	DetectionResult.jsx	Shows damage overlay, metrics, map pin, and actions for draft, submit, and PDF download.
Persist report	ReportsContext	Stores drafts and submitted reports in roadguard_user_reports in localStorage.
Visualize	Reports.jsx, MapView.jsx, Dashboard.jsx	Merges user reports with mock reports for tables, charts, filters, and Leaflet markers.

State and Persistence

Area	Current frontend source	Current persistence	Backend ready replacement
User session	AuthContext	localStorage key roadguard_user	Real auth session from backend or Supabase Auth.
Detection result	DetectionContext	React state only	ML inference API response linked to uploaded image and report id.
Reports	ReportsContext	localStorage key roadguard_user_reports	Reports table in PostgreSQL with PostGIS geometry for coordinates.
Images	Upload.jsx object URL	Temporary browser object URL	Object storage path plus metadata stored on report record.
Map data	mockReports plus user reports	Bundled mock data and localStorage	API query with geospatial filtering, severity filters, and pagination.
PDF export	reportUtils.downloadReportPDF	Browser print window	Client generated PDF or backend generated official report document.

Backend Integration Boundary

The frontend can keep its current route and page structure when a backend is added. The main change should be replacing context internals with service calls while preserving the page-level API used by the components.

Frontend contract	Future API or service	Why it matters
login, signup, logout, updateProfile	Authentication service	Moves mock auth out of localStorage and enables real identity, roles, and session expiry.
analyzeImage(imageUrl, gps, reportId)	Image upload plus detection inference endpoint	Connects captured road evidence to ML output and permanent storage.
saveDraft, submitReport, deleteReport	Reports API	Allows report lifecycle tracking, moderation, authority review, and multi-device access.
MapView report query	PostgreSQL/PostGIS search endpoint	Supports nearby potholes, bounding box queries, city filters, and severity-based map layers.
downloadReportPDF	Report export endpoint or client PDF renderer	Produces consistent downloadable documents for submitted complaints.

Recommended Frontend Direction

Keep the current React Router structure and provider boundaries because they already match the product workflow. As the project grows, introduce a small services layer between contexts and backend APIs, then migrate localStorage data to PostgreSQL/PostGIS through authenticated endpoints.

For the next version, the most useful frontend change is to create dedicated API modules for auth, reports, detection, uploads, and map queries. That will let the pages stay stable while persistence moves from the browser to the backend.